

関数

目次

関数とは

用途

書き方

実際に書いてみよう

オーバーロード

いろいろな便利関数 パイザ編

関数とは

ひとまとまりの処理に名前を付けたものです。

関数とは

() ←これがついてるやつが関数でした。

例えば① Console.WriteLine()は

与えられた値を文字に変換する



文字コードに変換する



OSに表示してほしい文字データを渡す



OSがターミナルにデータを渡す



ターミナルが受け取った文字を表示する

例えば② SceneManager.LoadScene()は

現在シーンのオブジェクトをすべてOnDisable()



現在シーンのオブジェクトをすべてOnDestroy()



未使用になったメモリの開放



次のシーンのデータをロード



次のシーンのデータをすべてAwake()



次のシーンのデータをすべてStart()

関数はすでに作ってあるものを使うことも
自分で作ることもできる。

関数の用途

可読性の向上

コードの使いまわし

イベント

可読性の向上

前回の 2 次元配列の Paiza の問題くらいから思ったと思います。
プログラム長いしどこで区切れてるかわからん。って

可読性の向上

```
1  using System;
2  using System.Linq;
3  using System.Collections.Generic;
4
5  class Program
6  {
7      static void Main()
8      {
9          string line = Console.ReadLine();
10         int[] data = line.Split(" ").Select(int.Parse).ToArray();
11         int xLength = data[1];
12         int yLength = data[0];
13
14         char[,] matrix = new char[yLength,xLength];
15         for(int y = 0; y < yLength; y++)
16         {
17             line = Console.ReadLine();
18             for(int x = 0; x < xLength; x++)
19             {
20                 matrix[y,x] = line[x];
21             }
22         }
23
24         line = Console.ReadLine();
25         data = line.Split(" ").Select(int.Parse).ToArray();
26         int changeX = data[1];
27         int changeY = data[0];
28
29         if(matrix[changeY,changeX] == '#')
30         {
31             matrix[changeY,changeX] = '.';
32         }
33         else if(matrix[changeY,changeX] == '.')
34         {
35             matrix[changeY,changeX] = '#';
36         }
37
38         for(int y = 0; y < yLength; y++)
39         {
40             for(int x = 0; x < xLength; x++)
41             {
42                 Console.Write(matrix[y,x]);
43             }
44             Console.WriteLine();
45         }
46     }
47 }
```

これどこで何書いてるか
わかりずらい不是吗？

可読性の向上

```
1 using System;
2 using System.Linq;
3 using System.Collections.Generic;
4
5 class Program
6 {
7     static int xLength;
8     static int yLength;
9     static char[,] matrix;
10    static int changeX;
11    static int changeY;
12
13    static void Main()
14    {
15        Input();
16        Reverse();
17        OutPut();
18    }
19
20    static void Input()
21    {
22        string line = Console.ReadLine();
23        int[] data = line.Split(" ").Select(int.Parse).ToArray();
24        xLength = data[1];
25        yLength = data[0];
26
27        matrix = new char[yLength,xLength];
28        for(int y = 0; y < yLength; y++)
29        {
30            line = Console.ReadLine();
31            for(int x = 0; x < xLength; x++)
32            {
33                matrix[y,x] = line[x];
34            }
35        }
36
37        line = Console.ReadLine();
38        data = line.Split(" ").Select(int.Parse).ToArray();
39        changeX = data[1];
40        changeY = data[0];
41    }
42
43    static void Reverse()
44    {
45        if(matrix[changeY,changeX] == '#')
46        {
47            matrix[changeY,changeX] = '.';
48        }
49        else if(matrix[changeY,changeX] == '.')
50        {
51            matrix[changeY,changeX] = '#';
52        }
53    }
54
55    static void OutPut()
56    {
57        for(int y = 0; y < yLength; y++)
58        {
59            for(int x = 0; x < xLength; x++)
60            {
61                Console.Write(matrix[y,x]);
62            }
63            Console.WriteLine();
64        }
65    }
66 }
```

谷山が見やすく直すところな感じ

可読性の向上

```
1 using System;
2 using System.Linq;
3 using System.Collections.Generic;
4
5 class Program
6 {
7     static int xLength;
8     static int yLength;
9     static char[,] matrix;
10    static int changeX;
11    static int changeY;
12
13    static void Main()
14    {
15        Input();
16        Reverse();
17        OutPut();
18    }
19
20    static void Input()
21    {
22        string line = Console.ReadLine();
23        int[] data = line.Split(" ").Select(int.Parse).ToArray();
24        xLength = data[1];
25        yLength = data[0];
26
27        matrix = new char[yLength,xLength];
28        for(int y = 0; y < yLength; y++)
29        {
30            line = Console.ReadLine();
31            for(int x = 0; x < xLength; x++)
32            {
33                matrix[y,x] = line[x];
34            }
35        }
36
37        line = Console.ReadLine();
38        data = line.Split(" ").Select(int.Parse).ToArray();
39        changeX = data[1];
40        changeY = data[0];
41    }
42
43    static void Reverse()
44    {
45        if(matrix[changeY,changeX] == '#')
46        {
47            matrix[changeY,changeX] = '.';
48        }
49        else if(matrix[changeY,changeX] == '.')
50        {
51            matrix[changeY,changeX] = '#';
52        }
53    }
54
55    static void OutPut()
56    {
57        for(int y = 0; y < yLength; y++)
58        {
59            for(int x = 0; x < xLength; x++)
60            {
61                Console.Write(matrix[y,x]);
62            }
63            Console.WriteLine();
64        }
65    }
66 }
```

```
static void Main()
{
    Input();
    Reverse();
    OutPut();
}
```

大事なのはいつも書いてるMain()の中身が少なくなっている事

これを見るとだいたいの処理の流れがわかる

関数

可読性の向上

```
1 using System;
2 using System.Linq;
3 using System.Collections.Generic;
4
5 class Program
6 {
7     static int xLength;
8     static int yLength;
9     static char[,] matrix;
10    static int changeX;
11    static int changeY;
12
13    static void Main()
14    {
15        Input();
16        Reverse();
17        OutPut();
18    }
19
20    static void Input()
21    {
22        string line = Console.ReadLine();
23        int[] data = line.Split(" ").Select(int.Parse).ToArray();
24        xLength = data[1];
25        yLength = data[0];
26
27        matrix = new char[yLength,xLength];
28        for(int y = 0; y < yLength; y++)
29        {
30            line = Console.ReadLine();
31            for(int x = 0; x < xLength; x++)
32            {
33                matrix[y,x] = line[x];
34            }
35        }
36
37        line = Console.ReadLine();
38        data = line.Split(" ").Select(int.Parse).ToArray();
39        changeX = data[1];
40        changeY = data[0];
41    }
42
43    {
44        if(matrix[changeY,changeX] == '#')
45        {
46            matrix[changeY,changeX] = '.';
47        }
48        else if(matrix[changeY,changeX] == '.')
49        {
50            matrix[changeY,changeX] = '#';
51        }
52    }
53
54    static void OutPut()
55    {
56        for(int y = 0; y < yLength; y++)
57        {
58            for(int x = 0; x < xLength; x++)
59            {
60                Console.Write(matrix[y,x]);
61            }
62            Console.WriteLine();
63        }
64    }
65
66 }
```

```
19
20 static void Input()
21 {
22     string line = Console.ReadLine();
23     int[] data = line.Split(" ").Select(int.Parse).ToArray();
24     xLength = data[1];
25     yLength = data[0];
26
27     matrix = new char[yLength,xLength];
28     for(int y = 0; y < yLength; y++)
29     {
30         line = Console.ReadLine();
31         for(int x = 0; x < xLength; x++)
32         {
33             matrix[y,x] = line[x];
34         }
35     }
36
37     line = Console.ReadLine();
38     data = line.Split(" ").Select(int.Parse).ToArray();
39     changeX = data[1];
40     changeY = data[0];
41 }
42
```

個々の中身も他の要素が無いから
見やすい

コードの使いまわし

これは今は気にしなくていいです。

同じ事を 2 回書く場合に使いまわせるよってものなのですが、
そんな場面あんまりないので気にしないで。

関数

イベント

実は皆さんすでに関数を作っているのですが
これに見覚えありますか

```
using UnityEngine;
```

```
public class NewBehaviourScript : MonoBehaviour
{
    void Start()
    {

    }

    void Update()
    {

    }
}
```

イベント

```
using UnityEngine;

public class NewBehaviourScript : MonoBehaviour
{
    void Start()
    {
        //オブジェクトが生成された時
    }

    void Update()
    {
        //フレームが更新された時
    }
}
```

```
class Program
{
    static void Main()
    {
        //プログラムが始まった時
    }
}
```

特定のタイミングで
呼び出される関数

関数

関数

Awake ()

呼ばれるタイミング

オブジェクト生成時

主な用途

初期化、GetComponent

OnEnable ()

オブジェクトが有効になった時

イベント登録

Start ()

最初の Update () の前

ゲーム開始時の処理

FixedUpdate ()

一定時間ごと（物理更新）

Rigidbody の操作

Update ()

毎フレーム

入力、移動

LateUpdate ()

Update () の後

カメラ追従

OnGUI ()

GUI描画時

IMGUI（現在はあまり使用しない）

OnDrawGizmos ()

Sceneビュー描画時

デバッグ表示

OnDrawGizmosSelected ()

オブジェクト選択中

範囲表示

OnCollisionEnter ()

Colliderが衝突した瞬間

当たり判定

OnCollisionStay ()

衝突中

接触処理

OnCollisionExit ()

衝突終了

接触終了処理

OnTriggerEnter ()

Triggerに入った瞬間

Trigger判定

OnTriggerStay ()

Trigger内にいる間

Trigger処理

OnTriggerExit ()

Triggerから出た時

Trigger終了処理

OnCollisionEnter2D ()

2D衝突開始

2D当たり判定

OnCollisionStay2D ()

2D衝突中

2D接触処理

OnCollisionExit2D ()

2D衝突終了

2D接触終了

OnTriggerEnter2D ()

2D Trigger侵入

2D Trigger判定

OnTriggerStay2D ()

2D Trigger内

2D Trigger処理

OnTriggerExit2D ()

2D Trigger退出

2D Trigger終了

OnMouseDown ()

マウスクリック

オブジェクト選択

OnMouseUp ()

マウスボタンを離す

ドラッグ終了

OnMouseDown ()

ドラッグ中

ドラッグ処理

OnMouseEnter ()

マウスが乗る

ホバー演出

OnMouseExit ()

マウスが離れる

ホバー終了

OnMouseOver ()

マウスが乗っている間

ホバー中処理

OnDisable ()

無効化時

イベント解除

OnDestroy ()

破棄時

後始末

OnApplicationFocus (bool)

フォーカス変更

一時停止対応

OnApplicationPause (bool)

アプリ一時停止

モバイル向け

OnApplicationQuit ()

アプリ終了時

セーブなど

Reset ()

コンポーネント追加時・Reset実行時

初期値設定

OnValidate ()

Inspectorの値変更時

値の検証・自動補正

書き方

戻り値の型 関数名(引数)

{

処理

}

関数

書き方-戻り値-

戻り値とは関数の呼び出し元に返却する値の事です。

```
string line = Console.ReadLine();
```

Console.ReadLine()は受け取った文字列が戻り値

```
string[] data = line.Split(" ");
```

Split()は分割した結果が戻り値

関数

書き方-戻り値-

```
float GetPi()  
{  
    return 3.141592f;  
}
```

例外として戻り値がいない場合、無い場合はvoidと書きます。
またその場合returnはいりません

関数

書き方-引数-

引数とは関数を動作させる際に関数に与える追加要素です。

書き方-引数-

Console.WriteLine()が毎回Hello Worldって表示したら何にも使えない

自由な文字を出力するためには呼び出すときなんて表示してほしいか指定したい。

関数を使うときに変数の中身入れてね！ っていう枠があるそれが引数です。

関数

書き方-引数-

```
float AddPi(int number)
{
    return number + 3.141592f;
}
```

引数は関数内で変数と同じように動作します。

書き方-引数-

```
void Main()  
{  
    int angle = 10;  
    AddPi(angle);  
}  
  
void AddPi(int number)  
{  
    number += 3.141592f;  
}
```

引数で受け取った変数を変更しても
元の値は変わらないので注意してください。

※一部例外がありますが先の授業で説明するため割愛します。

書き方-引数-

注意点

パイザやC言語の授業で関数をつける時は最初にstaticとつけてください。

理由はいつかの授業で解説するため割愛します。

関数

実際に書いてみよう

下記のプログラムに Output関数と AddNumbers関数を追加して正しく表示されるようにしましょう。

```
4 using System;
5 using System.Collections.Generic;
6 using System.Linq;
7
8 0 個の参照
9 class Program
10 {
11     0 個の参照
12     static void Main()
13     {
14         int[] numbers = { 5, 6, 7 };
15         Output(numbers);
16         int sum = AddNumbers(numbers);
17         Console.WriteLine(sum);
18     }
19
20
21
22 }
```



```
Microsoft Visual Studio デバッグ × + ▾
5
6
7
18
D:\001_講師\2026_バンタン 授業資料\C
ロセス 27692) は、コード 0 (0x0) で
デバッグが停止したときに自動的にコン
動的にコンソールを閉じる] を有効にし
このウィンドウを閉じるには、任意のキ
```

回答

```
4      ✓ class Program
5      {
6      0 個の参照
7      ✓ static void Main()
8      {
9          int[] numbers = { 5, 6, 7 };
10         Output(numbers);
11
12         int sum = AddNumbers(numbers);
13         Console.WriteLine(sum);
14     }
15
16     1 個の参照
17     ✓ static void Output(int[] numbers)
18     {
19         for (int i = 0; i < numbers.Length; i++)
20         {
21             Console.WriteLine(numbers[i]);
22         }
23     }
24
25     1 個の参照
26     ✓ static int AddNumbers(int[] numbers)
27     {
28         int sum = 0;
29         for (int i = 0; i < numbers.Length; i++)
30         {
31             sum += numbers[i];
32         }
33         return sum;
34     }
35 }
```

オーバーロード

このプログラムをみて違和感を覚えますか。

```
13     static void Main()  
14     {  
15         string text = "moji";  
16         int number = 100;  
17  
18         Console.WriteLine(text);  
19         Console.WriteLine(number);  
20     }  
21
```

オーバーロード

```
13     static void Main()  
14     {  
15         string text = "moji";  
16         int number = 100;  
17  
18         Console.WriteLine(text);  
19         Console.WriteLine(number);  
20     }  
21
```

← stringの引数
← int の引数

オーバーロード

このように引数の形が違う名前の同じ関数をつくることが出来ます。
これをオーバーロードといいます。

※紛らわしいですが、オーバーライドというものもあります。
別の授業でやるため割愛します。

いろいろな便利関数 パイザ編

数学処理

Math. Abs(数値)

絶対値

Math. Floor(数値)

少数切り捨て

Math. Max(数値、数値)

大きい方を返す

Math. Min(数値、数値)

小さい方を返す

Math. Pow(数値、数値)

累乗

Math. Sing(数値)

符号を返す

数値配列処理

. Sum()

配列すべての合計

. Average()

配列内の数値の平均

. Max()

配列内の中で最も大きい数

. Min()

配列内の中で最も小さい数

Array. Sort (配列)

配列のソート